Gradient Descent on Quadratics: Convergence Analysis

OMSCS ML | Optimization Unit

Table of Contents

1. Setup: The Quadratic Model
2. The Gradient of a Quadratic: Derivation via Differentials
3. The Workhorse Recursion
4. Decoupling into Independent 1D Processes
5. Convergence Condition: When Does GD Converge?
6. Rate Bound: How Fast Does It Converge?
7. Function Value Suboptimality Bound
8. The Zig-Zag Explained: 2D Algebra
9. Preconditioning and Newton's Method
10. Sources and Further Reading

---

1. Setup: The Quadratic Model

Why Quadratics?

Any smooth loss $f(\theta)$ looks quadratic in a small neighborhood around a local minimum. To see this precisely, Taylor-expand f around θ^* (a local minimizer):

$$f(\theta) \approx f(\theta^*) + \nabla f(\theta^*)^T(\theta - \theta^*) + \frac{1}{2}(\theta - \theta^*)^T \nabla^2 f(\theta^*)(\theta - \theta^*)$$

Since θ^* is a local minimizer, the gradient vanishes there: $\nabla f(\theta^*) = 0$. The linear term disappears, leaving:

$$f(\theta) - f(\theta^*) \approx \frac{1}{2}(\theta - \theta^*)^T H(\theta - \theta^*)$$

where $H := \nabla^2 f(\theta^*)$ is the Hessian at the optimum — a symmetric matrix (by Clairaut's theorem for smooth f), and positive semidefinite because θ^* is a local minimum. To see why PSD follows: for any direction $\mathbf{v}$, the second-order rate of change of f along $\mathbf{v}$ from θ^*

is $\mathbf{v}^T H \mathbf{v}$. Since θ^* is a local minimizer, f cannot decrease in any direction, so this second-order term must be ≥ 0 for every $\mathbf{v}$ — which is precisely the definition of $H \succeq 0$.

The upshot: analyzing GD on a quadratic is *exactly* analyzing GD on any smooth loss near a local minimum. The quadratic model is not an approximation we make for convenience — it's what the loss actually is, locally.

The Model and Its Shorthand

We study:

$$f(\theta) = f(\theta^*) + \frac{1}{2}(\theta - \theta^*)^T H (\theta - \theta^*)$$

Since $f(\theta^*)$ is a constant that doesn't affect gradients or GD dynamics, we subtract it and introduce the error vector $x := \theta - \theta^*$ (displacement from optimum):

$$f(x) = \frac{1}{2} x^T H x$$

where:

- $x \in \mathbb{R}^d$ is the error (we want $x \to 0$, i.e., $\theta \to \theta^*$)
- $H \in \mathbb{R}^{d \times d}$ is symmetric ($H = H^T$) and PSD ($H \succeq 0$)

Note on notation: the original parameters θ satisfy $x = \theta - \theta^*$, so $x = 0$ means $\theta = \theta^*$ — exactly at the optimum.

With the model established, the first thing we need is an explicit formula for the gradient of f in error coordinates — without it we cannot write down the GD update.

---

2. The Gradient of a Quadratic: Derivation via Differentials

Why Bother Deriving It?

We need $\nabla f(x) = Hx$. This is stated in most courses without proof, but it's worth deriving carefully once — both to verify it, and because the differential approach generalizes cleanly to other matrix calculus problems.

The Differential Approach

The **differential** of a scalar function f is a linear form df such that $df = (\nabla f)^T dx$ for any infinitesimal displacement dx. Given ∇f in this form, reading off ∇f is trivial: it's the vector that acts as the coefficient of dx.

Start from $f(x) = \frac{1}{2}x^T Hx$. Take the differential of both sides. The left side gives df. The right side requires differentiating $x^T Hx$, which is a product of three factors — we apply the product rule.

Product rule for differentials: For any functions $A(x)$ and $B(x)$:

$$d(A \cdot B) = (dA) \cdot B + A \cdot (dB)$$

Apply this to $x^T Hx = (x^T)(Hx)$, treating the first factor as x^T and the second as Hx:

$$d(x^T Hx) = (dx)^T (Hx) + x^T H(dx)$$

Here dx is the displacement, $(dx)^T$ is its transpose, and H is constant (doesn't depend on x).

So:

$$df = \frac{1}{2}\left[(dx)^T Hx + x^T H(dx)\right]$$

Now use symmetry $H = H^T$. Note that $(dx)^T Hx$ is a scalar, so it equals its own transpose:

$$(dx)^T Hx = \left[(dx)^T Hx\right]^T = x^T H^T (dx) = x^T H(dx)$$

The two terms are equal. So:

$$df = \frac{1}{2} \cdot 2 \cdot x^T H(dx) = x^T H(dx) = (Hx)^T dx$$

(In the last step we used $(x^T H)^T = H^T x = Hx$, again using $H = H^T$.)

Matching to the definition $df = (\nabla f)^T dx$:

$$\boxed{\nabla f(x) = Hx}$$

Translating back to θ-coordinates. Recall $x = \theta - \theta^*$, so $\nabla_\theta f(\theta) = H(\theta - \theta^*)$.

Key intuition: The gradient is *linear* in the error x. The Hessian H is the linear map that transforms the displacement from optimum into the gradient direction. This linearity is what makes quadratic analysis tractable — and it's exactly what makes the dynamics decouple, as we'll see.

3. The Workhorse Recursion

Gradient Descent in Error Coordinates

The GD update in θ-coordinates is:

$$\theta_{t+1} = \theta_t - \eta \nabla f(\theta_t)$$

Substitute $\nabla f(\theta_t) = H(\theta_t - \theta^*)$:

$$\theta_{t+1} = \theta_t - \eta H(\theta_t - \theta^*)$$

Subtract θ^* from both sides to convert to error coordinates $x_t = \theta_t - \theta^*$:

$$\theta_{t+1} - \theta^* = \theta_t - \theta^* - \eta H(\theta_t - \theta^*)$$

$$x_{t+1} = x_t - \eta H x_t = (I - \eta H) x_t$$

This is the **workhorse recursion**:

$$\boxed{x_{t+1} = (I - \eta H) x_t}$$

What It Means

Everything about GD on this quadratic is captured by repeated multiplication by the matrix $M := I - \eta H$. After t steps:

$$x_t = M^t x_0 = (I - \eta H)^t x_0$$

The error at step t is just M^t applied to the initial error. Convergence of GD $\Leftrightarrow M^t \to 0 \Leftrightarrow$ the eigenvalues of M all have absolute value < 1.

The reason the eigenvalues of H are the natural object to study is now clear: since $M = I - \eta H$, if $H\mathbf{q} = \lambda \mathbf{q}$ then $M\mathbf{q} = (1 - \eta\lambda)\mathbf{q}$. The eigenvectors are the same; the eigenvalues of M are $1 - \eta\lambda_i$.

---

4. Decoupling into Independent 1D Processes

Motivation

The recursion $x_{t+1} = (I - \eta H)x_t$ mixes all d coordinates together (through H). But since H is symmetric, its eigenvectors form an orthonormal basis, and in that basis the mixing disappears entirely.

Change of Coordinates

Decompose $H = Q\Lambda Q^T$ where $Q = [\mathbf{q}_1 \mid \cdots \mid \mathbf{q}_d]$ (orthonormal eigenvectors) and $\Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_d)$.

Define eigenbasis coordinates: $z_t := Q^T x_t$, equivalently $x_t = Qz_t$. This is just rotating the coordinate system to align with the eigenvectors of H — no stretching, no information loss.

Deriving the Decoupled Recursion

Start from the workhorse recursion and substitute $H = Q\Lambda Q^T$:

$$x_{t+1} = (I - \eta Q\Lambda Q^T)x_t$$

Multiply both sides on the left by Q^T:

$$Q^T x_{t+1} = Q^T(I - \eta Q\Lambda Q^T)x_t$$

Distribute Q^T on the right:

$$Q^T x_{t+1} = Q^T x_t - \eta Q^T Q\Lambda Q^T x_t$$

Use $Q^T x_t = z_t, Q^T x_{t+1} = z_{t+1}$, and $Q^T Q = I$ (orthonormality):

$$z_{t+1} = z_t - \eta\Lambda z_t = (I - \eta\Lambda)z_t$$

Since Λ is diagonal, this multiplication is component-wise:

$$\boxed{z_{t+1,i} = (1 - \eta\lambda_i)\, z_{t,i}}$$

Each coordinate i evolves completely independently of all others. Unrolling from the initial condition $z_{0,i}$:

$$z_{t,i} = (1 - \eta\lambda_i)^t\, z_{0,i}$$

What This Means

GD on a quadratic is just d independent 1D shrinkage processes running in parallel — one per eigendirection. In each direction i:

- The error component $z_{t,i}$ is multiplied by $(1 - \eta\lambda_i)$ every step
- λ_i controls the curvature in direction $\mathbf{q}_i$
- $z_{t,i}$ measures how far you are from the optimum along $\mathbf{q}_i$

The d-dimensional problem has been fully solved. All the interesting structure is now in the scalar factors $(1 - \eta\lambda_i)$.

Also notice the loss decouples in exactly the same way. Substituting $x_t = Qz_t$ into $f(x_t) = \frac{1}{2}x_t^T H x_t$:

$$f(x_t) = \frac{1}{2}(Qz_t)^T Q\Lambda Q^T(Qz_t) = \frac{1}{2}z_t^T \underbrace{Q^T Q}_{=I}\Lambda\underbrace{Q^T Q}_{=I}z_t = \frac{1}{2}z_t^T \Lambda z_t = \frac{1}{2}\sum_{i=1}^{d}\lambda_i z_{t,i}^2$$

The loss is a sum of d independent 1D quadratics. Each eigendirection contributes independently, with λ_i weighting the squared error in that direction.

So what? We now have exact closed-form expressions for both the error components $z_{t,i} = (1 - \eta\lambda_i)^t z_{0,i}$ and the loss $f(x_t) = \frac{1}{2}\sum_i \lambda_i z_{t,i}^2$. The d-dimensional problem has been completely solved in terms of d scalar sequences. This is what makes it possible to derive sharp, explicit conditions for convergence and tight rate bounds — which is exactly what the next two sections do.

---

5. Convergence Condition: When Does GD Converge?

The Question

From decoupling, $z_{t,i} = (1 - \eta\lambda_i)^t z_{0,i}$. For $z_{t,i} \to 0$ we need:

$$|1 - \eta\lambda_i| < 1$$

This must hold for every i with $\lambda_i > 0$ (directions with $\lambda_i = 0$ are flat — $z_{t,i} = z_{0,i}$ stays constant, neither growing nor shrinking).

Solving the Inequality

$$|1 - \eta\lambda_i| < 1$$

$$-1 < 1 - \eta\lambda_i < 1$$

Right inequality: $1 - \eta\lambda_i < 1 \Rightarrow -\eta\lambda_i < 0 \Rightarrow \eta\lambda_i > 0$. Since $\lambda_i > 0$, this just requires $\eta > 0$. ✓

Left inequality: $-1 < 1 - \eta\lambda_i$. Subtract 1 from both sides: $-2 < -\eta\lambda_i$. Multiply both sides by -1 (flipping the inequality): $2 > \eta\lambda_i$. Divide by $\lambda_i > 0$: $\eta < \frac{2}{\lambda_i}$.

So for each direction i, stability requires $\eta < 2/\lambda_i$. To satisfy all directions simultaneously, we need this for the worst case — the direction with the largest eigenvalue:

$$\boxed{0 < \eta < \frac{2}{\lambda_{\max}}}$$

This is the classical GD stability bound. It says: the learning rate is constrained by the direction of maximum curvature. The more curved the bowl is in its steepest direction, the smaller η must be.

What Happens When η Violates This Bound?

If $\eta > 2/\lambda_i$ for some direction i, then $|1 - \eta\lambda_i| > 1$: the error in direction i *grows* every step. GD diverges. This is why using too large a learning rate causes the loss to explode rather than converge.

We now know *when* GD converges. The next question is *how fast* — and how the answer depends on the geometry of H.

---

6. Rate Bound: How Fast Does It Converge?

Setting Up the Bound

Assume $H \succ 0$ (all $\lambda_i > 0$) and η is in the stable range. We want to bound $\|x_t\|_2$, the Euclidean distance from the optimum after t steps.

Since $x_t = Qz_t$ and Q is orthonormal, Q preserves lengths:

$$\|x_t\|_2 = \|Qz_t\|_2 = \|z_t\|_2$$

(Orthonormal matrices are isometries — they rotate but don't stretch. Formally: $\|Qz\|^2 = (Qz)^T(Qz) = z^TQ^TQz = z^Tz = \|z\|^2$.)

So $\|x_t\|_2^2 = \|z_t\|_2^2 = \sum_i z_{t,i}^2$. Substituting $z_{t,i} = (1-\eta\lambda_i)^t z_{0,i}$:

$$\|x_t\|_2^2 = \sum_{i=1}^{d} (1-\eta\lambda_i)^{2t} z_{0,i}^2$$

Introducing the Contraction Factor

Define the **worst-case contraction factor** ρ as:

$$\rho := \max_i |1-\eta\lambda_i|$$

This is the largest per-step shrinkage factor across all eigendirections. Since $|1-\eta\lambda_i| \le \rho$ for every i, squaring both sides gives $(1-\eta\lambda_i)^2 \le \rho^2$, and raising to the t-th power:

$$(1-\eta\lambda_i)^{2t} \le \rho^{2t}$$

Substituting into the norm bound:

$$\|x_t\|_2^2 \le \sum_{i=1}^{d} \rho^{2t} z_{0,i}^2 = \rho^{2t} \sum_{i=1}^{d} z_{0,i}^2 = \rho^{2t}\|z_0\|_2^2 = \rho^{2t}\|x_0\|_2^2$$

Taking square roots:

$$\boxed{\|x_t\|_2 \le \rho^t \|x_0\|_2}$$

The error contracts geometrically at rate ρ per step. After t steps, at most a fraction ρ^t of the initial error remains.

Choosing η to Minimize ρ

Now we want to find the η that makes ρ as small as possible.

With $H \succ 0$, eigenvalues lie in $[\lambda_{\min}, \lambda_{\max}]$. The function $g(\lambda) = |1-\eta\lambda|$ is V-shaped in λ: it equals zero at $\lambda = 1/\eta$ and grows linearly on both sides. Its maximum over $[\lambda_{\min}, \lambda_{\max}]$ is therefore at one of the two endpoints:

$$\rho(\eta) = \max\{|1-\eta\lambda_{\min}|,\ |1-\eta\lambda_{\max}|\}$$

To minimize over η: notice that increasing η decreases $|1 - \eta\lambda_{\min}|$ (pulls it toward zero) but increases $|1 - \eta\lambda_{\max}|$ (pushes it past zero and away). So the two terms move in opposite directions as η changes, and the minimax optimum is where they are equal. We therefore want to find η satisfying:

$$|1 - \eta\lambda_{\min}| = |1 - \eta\lambda_{\max}|$$

Identifying the correct regime. When η is very small ($\eta \ll 1/\lambda_{\max}$), both $1 - \eta\lambda_{\min}$ and $1 - \eta\lambda_{\max}$ are positive, and $\lambda_{\max}$ still gives the larger term — so we want to increase η. When $\eta > 1/\lambda_{\min}$, both terms are negative and $\lambda_{\max}$ again dominates — we want to decrease η. The optimal must therefore lie in the interval $1/\lambda_{\max} < \eta < 1/\lambda_{\min}$, where $1 - \eta\lambda_{\min} > 0$ and $1 - \eta\lambda_{\max} < 0$. In this regime the balance condition becomes:

$$1 - \eta\lambda_{\min} = -(1 - \eta\lambda_{\max})$$

$$1 - \eta\lambda_{\min} = -1 + \eta\lambda_{\max}$$

$$2 = \eta(\lambda_{\min} + \lambda_{\max})$$

$$\eta^* = \frac{2}{\lambda_{\min} + \lambda_{\max}}$$

Now compute ρ^*. Since $\eta^* > 1/\lambda_{\max}$, the quantity $1 - \eta^*\lambda_{\max}$ is negative, so we need the absolute value:

$$\rho^* = |1 - \eta^*\lambda_{\max}| = \left|1 - \frac{2\lambda_{\max}}{\lambda_{\min} + \lambda_{\max}}\right| = \left|\frac{\lambda_{\min} + \lambda_{\max} - 2\lambda_{\max}}{\lambda_{\min} + \lambda_{\max}}\right| = \left|\frac{\lambda_{\min} - \lambda_{\max}}{\lambda_{\min} + \lambda_{\max}}\right| = \frac{\lambda_{\max} - \lambda_{\mathrm{r}}}{\lambda_{\max} + \lambda_{\mathrm{r}}}$$

Dividing numerator and denominator by $\lambda_{\min}$ and writing $\kappa = \lambda_{\max}/\lambda_{\min}$:

$$\boxed{\rho^* = \frac{\kappa - 1}{\kappa + 1}}$$

And therefore, with optimal learning rate:

$$\|x_t\|_2 \leq \left(\frac{\kappa - 1}{\kappa + 1}\right)^t \|x_0\|_2$$

How Many Steps to Converge?

To reduce the error by a factor of ϵ (i.e., reach $\|x_t\| \leq \epsilon \|x_0\|$), we need $\rho^{*t} \leq \epsilon$, i.e.:

$$t \geq \frac{\log(1/\epsilon)}{\log(1/\rho^*)}$$

For large κ, $\rho^* = (\kappa - 1)/(\kappa + 1) \approx 1 - 2/\kappa$, so $\log(1/\rho^*) = -\log(\rho^*) \approx 2/\kappa$ (using $\log(1 - x) \approx -x$ for small $x = 2/\kappa$). Therefore:

$$t \gtrsim \frac{\kappa}{2} \log \frac{1}{\epsilon}$$

The required number of steps scales as $O(\kappa \log 1/\epsilon)$. For a fixed target accuracy ϵ, this is linear in the condition number. A well-conditioned problem ($\kappa \approx 1$) needs far fewer steps than an ill-conditioned one ($\kappa \gg 1$).

---

7. Function Value Suboptimality Bound

Setup

The error norm bound from Section 6 tells us how far we are from θ^* in parameter space. A complementary bound is on the *function value* suboptimality: how much larger than $f(\theta^*)$ is $f(\theta_t)$?

Since $f(x^*) = f(0) = 0$ in our model, this is just $f(x_t)$.

Deriving the Bound

From Section 4:

$$f(x_t) = \frac{1}{2} \sum_{i=1}^{d} \lambda_i z_{t,i}^2$$

Substitute $z_{t,i} = (1 - \eta\lambda_i)^t z_{0,i}$:

$$f(x_t) = \frac{1}{2} \sum_{i=1}^{d} \lambda_i (1 - \eta\lambda_i)^{2t} z_{0,i}^2$$

Upper-bound in two steps. First, $(1-\eta\lambda_i)^{2t} \le \rho^{2t}$ (same argument as Section 6). Second, $\lambda_i \le \lambda_{\max}$. Combining:

$$f(x_t) \le \frac{1}{2}\sum_{i=1}^{d} \lambda_{\max}\rho^{2t} z_{0,i}^2 = \frac{\lambda_{\max}}{2}\rho^{2t}\|z_0\|_2^2 = \frac{\lambda_{\max}}{2}\rho^{2t}\|x_0\|_2^2$$

$$\boxed{f(x_t) \le \frac{\lambda_{\max}}{2}\rho^{2t}\|x_0\|_2^2}$$

A Relative Bound in Terms of $f(x_0)$

The bound above requires knowing $\|x_0\|$. We can eliminate this by expressing it in terms of $f(x_0)$, yielding a relative bound.

From the loss formula:

$$f(x_0) = \frac{1}{2}\sum_i \lambda_i z_{0,i}^2 \ge \frac{\lambda_{\min}}{2}\sum_i z_{0,i}^2 = \frac{\lambda_{\min}}{2}\|z_0\|_2^2 = \frac{\lambda_{\min}}{2}\|x_0\|_2^2$$

Rearranging: $\|x_0\|_2^2 \le \frac{2}{\lambda_{\min}} f(x_0)$. Substituting into the absolute bound:

$$f(x_t) \le \frac{\lambda_{\max}}{2}\rho^{2t} \cdot \frac{2}{\lambda_{\min}} f(x_0) = \frac{\lambda_{\max}}{\lambda_{\min}}\rho^{2t} f(x_0) = \kappa\rho^{2t} f(x_0)$$

$$\boxed{f(x_t) \le \kappa\rho^{2t} f(x_0)}$$

So what? This bound makes the role of κ explicit in two places: as a multiplicative constant upfront, and inside ρ^{2t} in the exponent. Both degrade with κ. The κ prefactor is an artifact of the looseness in the bound (we over-bounded λ_i by $\lambda_{\max}$ everywhere), so this is not a tight constant. But it correctly captures the qualitative dependence: high condition number → both a larger initial "penalty" and slower geometric decay.

The rate bounds in Sections 6 and 7 tell us that convergence degrades with κ — but they don't explain *why* intuitively. The next section makes that concrete via the exact 2D dynamics, showing precisely how κ produces the characteristic zig-zag trajectory.

---

8. The Zig-Zag Explained: 2D Algebra

The Observed Phenomenon

When gradient descent is applied to an ill-conditioned problem, the iterates exhibit a characteristic **zig-zag** pattern: they oscillate back and forth across the steep "walls" of the loss surface while barely advancing along the shallow "floor." The decoupled recursion gives an exact algebraic explanation.

Setting Up the 2D Case

Work in 2D with two eigendirections: eigenvalue λ_1 (steep direction) and λ_2 (shallow direction), with $\lambda_1 \gg \lambda_2$. In eigen-coordinates:

$$z_{t+1,1} = (1 - \eta\lambda_1)z_{t,1}, \qquad z_{t+1,2} = (1 - \eta\lambda_2)z_{t,2}$$

For stability, we need $\eta < 2/\lambda_1$. A natural choice is $\eta \approx 1/\lambda_1$ (half the stability limit, a typical conservative choice).

Progress in the Steep Direction (λ_1)

With $\eta = 1/\lambda_1$:

$$1 - \eta\lambda_1 = 1 - 1 = 0 \implies z_{t+1,1} = 0$$

The steep direction is eliminated in a single step. In practice η is slightly less than $1/\lambda_1$, so $(1 - \eta\lambda_1)$ is small and positive. Either way, the steep direction converges very quickly.

Progress in the Shallow Direction (λ_2)

With the same η, the shrinkage factor in the shallow direction is:

$$1 - \eta\lambda_2 \approx 1 - \frac{\lambda_2}{\lambda_1} = 1 - \frac{1}{\kappa}$$

where $\kappa = \lambda_1/\lambda_2$. For large κ, this is very close to 1 — the shallow direction barely shrinks. After t steps:

$$z_{t,2} = \left(1 - \frac{1}{\kappa}\right)^t z_{0,2}$$

To reduce $z_{t,2}$ by a factor of $1/e$, we need $(1 - 1/\kappa)^t \approx e^{-t/\kappa} = 1/e$, so $t \approx \kappa$. The shallow direction requires $O(\kappa)$ steps — while the steep direction took just $O(1)$.

This mismatch is the zig-zag. Progress happens fast in steep directions and painfully slowly in shallow directions. In a 2D picture, the iterates jump rapidly to near the valley floor (steep z_1 eliminated), then creep along it (shallow z_2 barely moves).

When Zig-Zag Involves Sign Flipping

The zig-zag is most dramatic — and most visually obvious — when the shrinkage factor in the steep direction is **negative**:

$$-1 < 1 - \eta\lambda_1 < 0 \iff \frac{1}{\lambda_1} < \eta < \frac{2}{\lambda_1}$$

When this happens, the sign of $z_{t,1}$ flips every step: the iterate alternates between two sides of the valley in the steep direction. Combined with slow monotone decay in the shallow direction, this produces the characteristic bouncing-across-the-ravine trajectory.

Concretely: if $z_{0,1} > 0$ and $1 - \eta\lambda_1 < 0$, then $z_{1,1} < 0$, $z_{2,1} > 0$, etc. In original θ-coordinates, the optimizer oscillates across the steep axis while slowly advancing along the shallow one.

Bottom line: κ forces η to be small (for stability in steep directions), which makes progress in flat directions painfully slow. The zig-zag is not a flaw in GD — it's the inevitable consequence of using a single step size on a non-uniform curvature landscape.

---

9. Preconditioning and Newton's Method

The Root Cause

All the difficulties with GD trace to one source: the eigenvalues of H are spread out. The step size is constrained by $\lambda_{\max}$, but convergence in the flat directions is limited by $\lambda_{\min}$. The ratio $\kappa = \lambda_{\max}/\lambda_{\min}$ is the gap between these two requirements.

The fix: transform the problem so the effective curvature is uniform across all directions. If the modified problem has condition number ≈ 1, GD on the modified problem converges in $O(1)$ steps.

Preconditioned Gradient Descent (PGD)

Instead of the standard GD update $x_{t+1} = x_t - \eta H x_t$, introduce a **preconditioner** — a symmetric PD matrix $P \succ 0$ — and use:

$$x_{t+1} = x_t - \eta P^{-1}\nabla f(x_t) = x_t - \eta P^{-1} H x_t = (I - \eta P^{-1} H)x_t$$

The dynamics are now governed by the matrix $P^{-1}H$ instead of H. If we choose P to approximate H, the eigenvalues of $P^{-1}H$ are concentrated near 1, and the effective condition number drops.

Making the Analysis Symmetric

$P^{-1}H$ is not generally symmetric, which complicates analysis. We resolve this by noting that $P^{-1}H$ is **similar** to the symmetric matrix:

$$\tilde{H} := P^{-1/2}HP^{-1/2}$$

Here $P^{-1/2}$ is the symmetric square root of P^{-1} — since $P \succ 0$, this exists and is also symmetric PD. To verify $\tilde{H}$ is symmetric:

$$\tilde{H}^T = (P^{-1/2}HP^{-1/2})^T = (P^{-1/2})^T H^T (P^{-1/2})^T = P^{-1/2}HP^{-1/2} = \tilde{H}$$

using $H = H^T$ and $(P^{-1/2})^T = P^{-1/2}$ (since $P^{-1/2}$ is itself symmetric).

To verify $P^{-1}H$ and $\tilde{H}$ have the same eigenvalues — i.e., that they are similar matrices — write:

$$P^{-1}H = P^{-1/2}(P^{-1/2}HP^{-1/2})P^{1/2} = P^{-1/2}\tilde{H}P^{1/2}$$

This expresses $P^{-1}H$ as $P^{-1/2}\tilde{H}(P^{-1/2})^{-1}$ — they are conjugate via the invertible matrix $P^{-1/2}$, which is the definition of similarity. Similar matrices share the same characteristic polynomial: $\det(P^{-1}H - \lambda I) = \det(P^{-1/2}(\tilde{H} - \lambda I)P^{1/2}) = \det(P^{-1/2})\det(\tilde{H} - \lambda I)\det(P^{1/2}) = \det(\tilde{H} - \lambda I)$, where the last equality uses $\det(P^{-1/2})\det(P^{1/2}) = 1$. Same characteristic polynomial means identical eigenvalues.

Practical implication: the convergence rate of PGD is governed by $\kappa(\tilde{H}) = \kappa(P^{-1/2}HP^{-1/2})$, not $\kappa(H)$. The goal is to choose P so that $\kappa(P^{-1/2}HP^{-1/2}) \ll \kappa(H)$.

The Ideal Preconditioner: $P = H$

For this to work, H must be invertible — i.e., $H \succ 0$ (PD), not merely PSD. We assume this for the rest of the section. If we could choose $P = H$ itself, then:

$$\tilde{H} = H^{-1/2}HH^{-1/2} = I$$

All eigenvalues of $\tilde{H}$ are 1, so $\kappa(\tilde{H}) = 1$. The PGD update becomes:

$$x_{t+1} = (I - \eta H^{-1}H)x_t = (I - \eta I)x_t = (1 - \eta)x_t$$

Pick $\eta = 1$:

$$x_{t+1} = 0$$

One step to the exact optimum. The preconditioner has perfectly equalized the curvature, and a single step of size 1 lands exactly at $x = 0$ (i.e., $\theta = \theta^*$).

Why is $P = H$ the "ideal"? Because $H^{-1}H = I$ replaces all eigenvalues of the dynamics with 1 — the curvature is perfectly uniform in every direction.

Newton's Method

The update $x_{t+1} = x_t - H^{-1}\nabla f(x_t) = x_t - H^{-1}Hx_t = x_t - x_t = 0$ is exactly **Newton's method**: use the inverse Hessian as the preconditioner. For the quadratic model, Newton converges in a single step.

Let's verify this in θ-coordinates. Newton's update is:

$$\theta_{t+1} = \theta_t - [\nabla^2 f(\theta_t)]^{-1}\nabla f(\theta_t)$$

For our quadratic, $\nabla^2 f(\theta) = H$ everywhere (Hessian of a quadratic is constant), and $\nabla f(\theta) = H(\theta - \theta^*)$. So:

$$\theta_{t+1} = \theta_t - H^{-1}H(\theta_t - \theta^*) = \theta_t - (\theta_t - \theta^*) = \theta^*$$

One step, exact convergence.

Why Don't We Always Use Newton?

If Newton's method solves quadratics in one step, why use GD at all? Two reasons:

1. **Cost.** Each Newton step requires computing $H^{-1}\nabla f$, which involves solving a $d \times d$ linear system — $O(d^3)$ in general. GD costs $O(d)$ per step (just compute the gradient). For large neural networks ($d \sim 10^8$), Newton is completely infeasible.
2. **Generalization beyond quadratics.** For non-quadratic losses, the Hessian changes at every point. Newton still works (it uses the local Hessian), but the convergence guarantees are local and computing the Hessian itself is expensive. In the stochastic/mini-batch setting, Hessian estimates are noisy and hard to invert reliably.

Preconditioning sits between the two extremes: use a matrix P that approximates H cheaply (e.g., its diagonal, or a low-rank approximation), gaining much of the benefit of

Newton at a fraction of the cost. Optimizers like Adam can be viewed as adaptive diagonal preconditioners.

Key Takeaways

1. **The quadratic model is exact locally.** Near any local minimum of a smooth loss, the loss is a quadratic form $\frac{1}{2}x^T Hx$ in the error vector.
2. **GD reduces to the recursion $x_{t+1} = (I - \eta H)x_t$.** Everything about GD on a quadratic is multiplication by this one matrix.
3. **Diagonalization decouples the d-dimensional problem into d independent 1D problems.** Each eigendirection shrinks independently by factor $(1 - \eta\lambda_i)$ per step.
4. **The convergence condition is $\eta < 2/\lambda_{\max}$.** Violating this causes divergence in the steepest direction.
5. **The optimal rate is $\rho^* = (\kappa - 1)/(\kappa + 1)$.** This requires $O(\kappa)$ steps to converge and is derived by balancing the two endpoint shrinkage factors.
6. **Zig-zag is the algebraic consequence of using one η on a non-uniform curvature landscape.** Steep directions are overcontrolled; shallow directions are undercontrolled.
7. **Preconditioning rescales the problem.** The ideal preconditioner $P = H$ (Newton's method) achieves $\kappa = 1$ and one-step convergence, at $O(d^3)$ cost per step.

---

10. Sources and Further Reading

The material in this document is standard in optimization; the derivations synthesize classical results from several sources.

- **Nocedal & Wright, *Numerical Optimization***, Ch. 2–3. The workhorse recursion, condition number, and rate bounds for GD on quadratics. The analysis here follows their general approach.
- **Boyd & Vandenberghe, *Convex Optimization***, §9.1–9.3. The descent lemma, convergence rates, and Newton's method. The preconditioning interpretation of Newton is treated in §9.5.
- **Polyak, *Introduction to Optimization* (1987).** The optimal step-size derivation for quadratics (the $\eta^* = 2/(\lambda_{\min} + \lambda_{\max})$ result) is classical and attributed to this line of work.

- For the **zig-zag visualization and geometric intuition**, Goodfellow, Bengio & Courville, *Deep Learning*, Ch. 8, gives a nice discussion of ill-conditioning in the deep learning context.
- The **preconditioning and Newton connection** is treated rigorously in Nocedal & Wright Ch. 6 (quasi-Newton) and Boyd & Vandenberghe §9.5. The idea that Adam approximates a diagonal preconditioner is discussed in Kingma & Ba (2015), *Adam: A Method for Stochastic Optimization*.